

Prise en main du kit de développement IoT P-NUCLEO-LRWAN3 de ST Microelectronics

Création d'un objet connecté LoRa :

Conception d'un firmware en langage C

Lecture de capteurs Envoi de données Réception et interprétation depuis le Cloud

Table des matières

Objectifs.....	3
Prérequis et modalités.....	3
Présentation du matériel.....	4
1. Étapes préparatoires.....	5
2. Créer un nouveau projet orienté LoRa.....	6
Versionner le projet sur votre compte GitHub.....	8
Configurer le projet.....	9
Considérer les warnings comme des erreurs.....	9
3. Compiler et exécuter pour la première fois.....	10
4. Mise en œuvre d'un capteur, formatage des données.....	14
Repérer et intégrer l'en-tête du capteur de température.....	14
Instancier sa structure de travail.....	14
Initialiser le capteur.....	14
Lire le capteur, vérifier la lecture.....	14
5. Convertir les données en valeurs à virgule fixe.....	15
Générer les données envoyées par votre objet LoRa.....	16
6. Envoyer vos données au contrôleur LoRa.....	17
7. Faire transiter vos données vers le Cloud.....	18
Enregistrement de votre carte dans le cloud.....	18
Recevoir vos données depuis le cloud.....	19
8. Recommencez avec les capteurs d'humidité et de pression.....	20

Objectifs

A la fin de ce TP, vous aurez réussi à faire un objet connecté LoRa qui :

- se connecte à un réseau longue portée basse consommation (**Long Range** aka **LoRa**)
- envoie des mesures de plusieurs capteurs sur une passerelle avoisinante (jusqu'à 15 km)
- mesures immédiatement visibles et exploitables depuis le cloud !

Prérequis et modalités

Ce TP doit être fait sur **votre propre ordinateur portable**.
Évitez les ordinateurs de l'école, **ça ne fonctionnera pas**.

Pour rendre vos résultats, il me faudra, par groupe:

- Un compte-rendu
- Le code du projet.

Vous pouvez me rendre le tout dans un zip contenant :

- le compte-rendu et le dossier du code,
- ou le compte-rendu et le lien GitHub de votre code (dans le compte-rendu 😊).

On vous communiquera ultérieurement le moyen de transférer votre archive.

Au sujet du contenu de votre rapport:

- des screenshots pertinents sont attendus
- Vous allez écrire du code sur une plate-forme limitée. Donc la propreté et l'utilisation de bonnes pratiques sont fortement avisés

Soyez sûr.es à 200% de lire le sujet de TP de A à Z avant que de commencer.
Imaginez que ce texte clignote et fait plein d'étranges bruits d'alertes Windows
[\[cliquez ici !\]](#)

Présentation du matériel

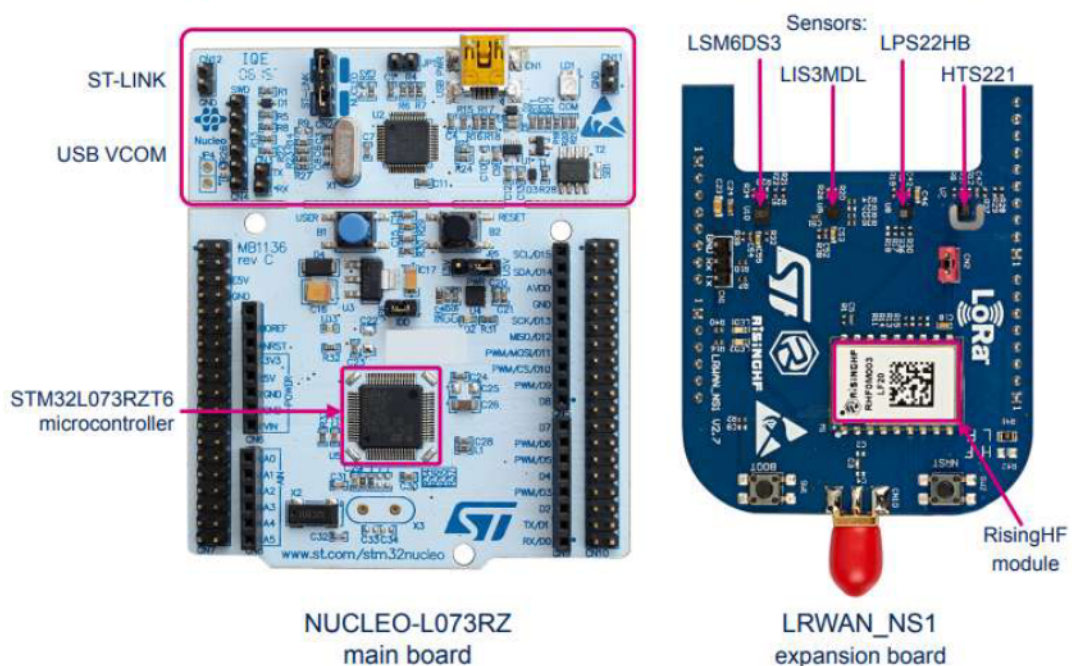
La carte que vous avez dans les mains est la **P-NUCLEO-LRWAN3** de **ST Microelectronics**.

Le modèle est : **NUCLEO-L073RZ**.

!! RETENEZ BIEN CE NOM SVP !!

!! C'EST IMPORTANT POUR SAVOIR OÙ RAJOUTER VOTRE CODE !!

Figure 9. STM32 Nucleo LoRaWAN® sensor device (P-NUCLEO-LRWAN3)



Cette carte se veut une rivale de l'Arduino (elle est pin-compatible) mais en plus polyvalente.

Celle-ci est surmontée d'un shield **LoRa LRWAN-NS1** de chez RisingHF. Nous allons travailler avec ce sandwich de cartes.

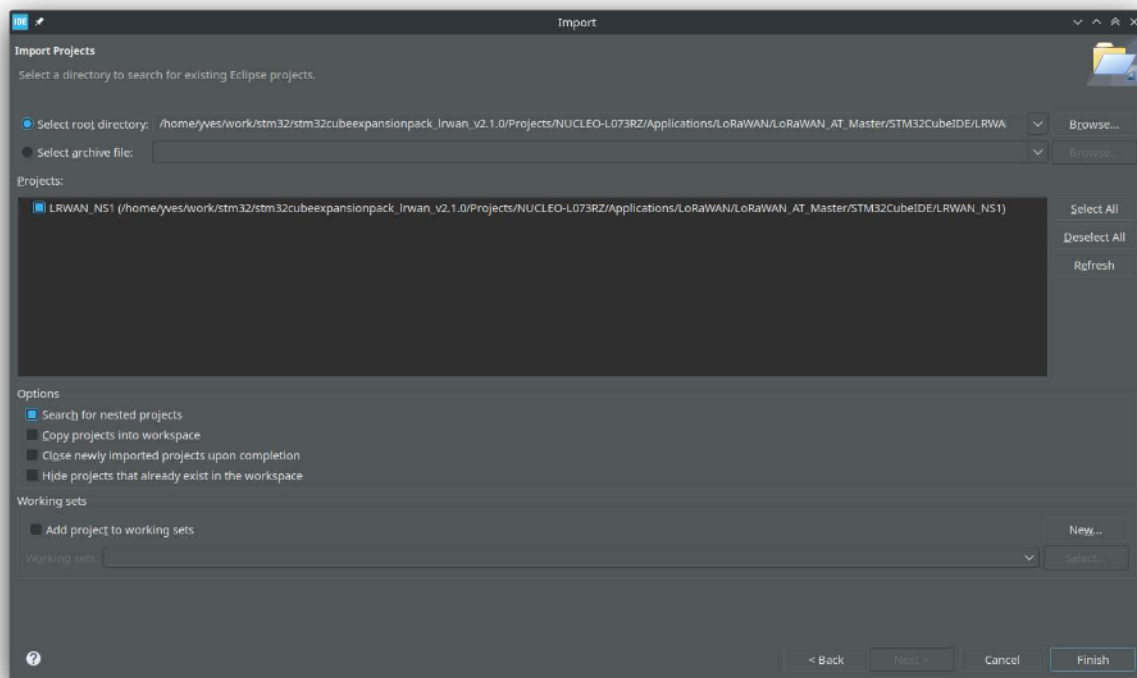
1. Étapes préparatoires

1. Créez un compte sur le site de ST Microelectronics : <https://my.st.com/cas/login>
2. Téléchargez l'environnement de développement **STM32CubeIDE version 2.20** :
<https://www.st.com/en/development-tools/stm32cubeide.html>
3. Installez-le.
Soyez d'accord avec tous ses termes et conditions sans même les lire 😊
4. **IMPORTANT** : l'installateur peut vous proposer d'installer des pilotes J-Link dans votre système : **acceptez sa demande**
5. Téléchargez la documentation du **P-NUCLEO-LRWAN3 Starter Pack** ici:
https://www.st.com/resource/en/user_manual/um2587-getting-started-with-the-pnucleolrwan2-and-pnucleolrwan3-starter-packs-stmicroelectronics.pdf
6. Téléchargez l'extension LRWAN pour STM32CubeIDE :
stm32cubeexpansionpack_lrwan_v2.1.0
L'enseignant vous fournira un lien.
Cette extension permet à STM32CubeIDE de fonctionner avec plusieurs cartes LoRa dont la **P-NUCLEO-LRWAN3** (la vôtre)
7. Décompressez-la dans un endroit sûr de votre ordinateur :
 - Loin de tout Onedrive 💣
 - Dans un chemin sans **es p ac es** ni **áccéntués**
 - Ceci vous fera gagner un temps considérable en vous évitant un bon nombre de problèmes bloquants 😊

2. Créer un nouveau projet orienté LoRa

Nous allons créer un projet LoRa basé sur le pack d'expansion LRWAN.

0. Lancez STM32CubeIDE
1. Fermez la fenêtre « **information center** »
2. STM32CubeIDE est (encore une fois) un dérivé d'Eclipse.
Il travaille avec un workspace, c'est-à-dire le dossier concentrant l'ensemble de vos projets STM32CubeIDE. Laissez-le vous créer / confirmer l'utilisation d'un workspace au chemin qu'il propose
3. Connectez-vous à votre compte ST Micro depuis STM32CubeIDE
4. Depuis le menu **File**, allez dans **Import** → **General** → **existing projects into workspace**
5. Ouvrez **exactement** le chemin de projet suivant :
stm32cubeexpansionpack_lrwan_v2.1.0/Projects/NUCLEO-L073RZ/Applications/LoRaWAN/LoRaWAN_AT_Master/STM32CubeIDE/LRWAN_NS1
6. Vous devriez avoir une fenêtre de demande de confirmation comme ceci :



Le projet LRWAN_NS1 sera coché d'office, vous n'avez qu'à cliquer sur Next.
Attention, pour voir que vous avez bien importé le projet, cliquez sur le menu

Window → Show View → Project Explorer.

Vous devriez voir votre projet **LRWAN_NS1** sur laquelle commencer à étudier.

7. **Félicitations !** Vous avez maintenant une base de projet LoRa prête à fonctionner avec votre carte.
8. Il reste cependant un bon nombre de choses à faire pour que la chaîne de traitement LoRa soit active et valide, de la lecture des capteurs à l'envoi des données.
9.  **Note importante**  :
Parce que vous avez fait un import de projet, l'ensemble des modifications que vous y apporterez seront écrites **dans le dossier du pack d'extension**

Optionnel mais fortement recommandé : versionner le projet sur votre compte GitHub

1. Ceci vous permet de travailler sereinement sur votre projet en le versionnant à chaque étape critique de votre développement (comme pour Android). Vous pouvez également de revenir sur vos pas au cas où quelque chose dans votre code ne fonctionne plus.
2. L'intégration de **git** dans STM32CubeIDE n'est pas native. Vous pouvez utiliser **git** en ligne de commande (comme des pros que vous êtes)
3. Dès maintenant, avec le terminal de votre système, allez dans le répertoire **du code qu'on vous a fourni et que vous avez décompressé.**
4. Tapez **git init**
5. Puis **git add .** (avec le '.' à la fin de la commande)
6. Trouvez-vous des initiales. Par exemple si votre binôme est composé de Bruce Willis et Samuel L. Jackson, vos initiales seront **BW_SJ**.
7. Faites un commit avec **git commit -m "commit initial groupe BW_SJ"**
8. Puis continuez sur le mode d'emploi d'utilisation de GitHub ici :
<https://docs.github.com/en/get-started/importing-your-projects-to-github/importing-source-code-to-github/adding-locally-hosted-code-to-github>
Votre répertoire de code GitHub devra porter **exactement** le nom suivant :
LRWAN_NS1_BW_SJ (adaptez avec vos propres initiales).

Configurer le projet

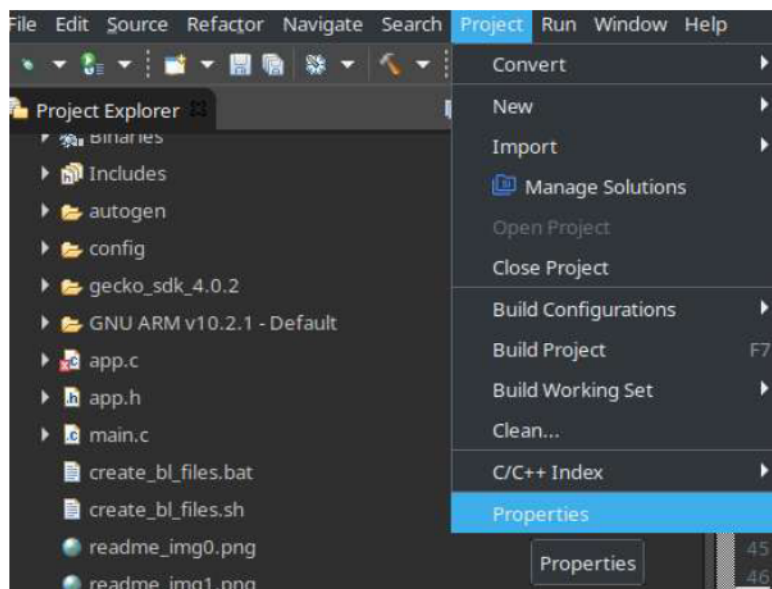
Nous allons maintenant configurer le projet à son démarrage pour partir sur de bonnes bases.

Considérer les warnings comme des erreurs

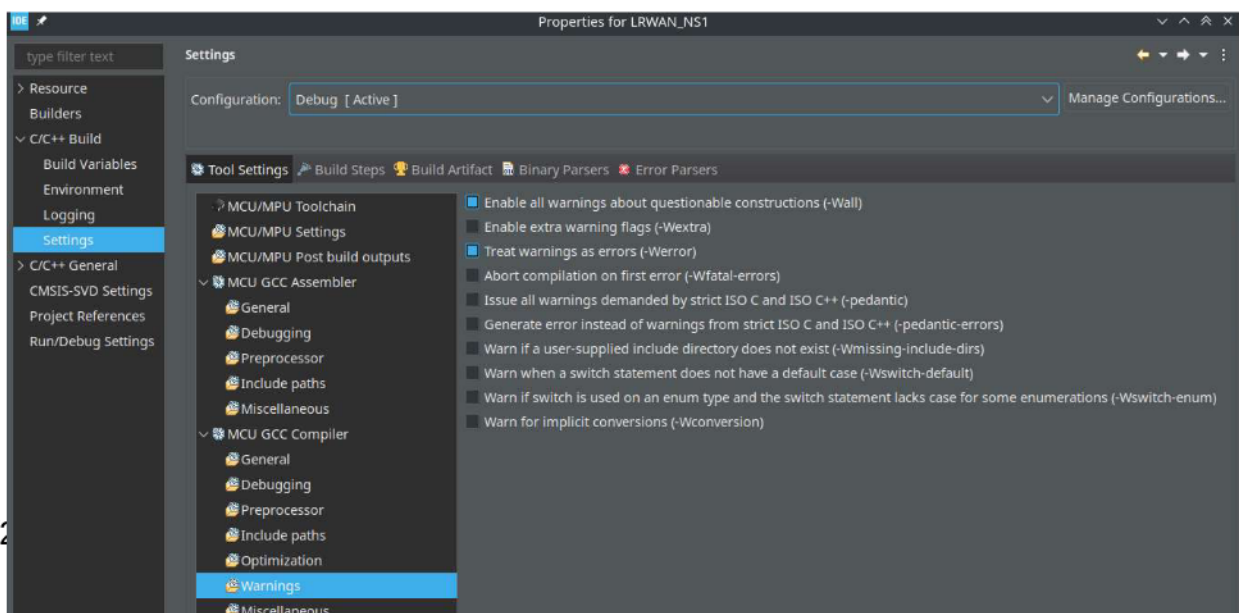
Tel quel, le compilateur est configuré de manière laxiste et passe sur les warnings alors que cela peut cacher une faute de programmation majeure.

Nous allons configurer les warnings comme des erreurs, c'est un mode de programmation plus sûr, surtout dans le contexte d'un logiciel embarqué IoT.

1. Dans le menu **Project**, cliquez sur **Properties**.

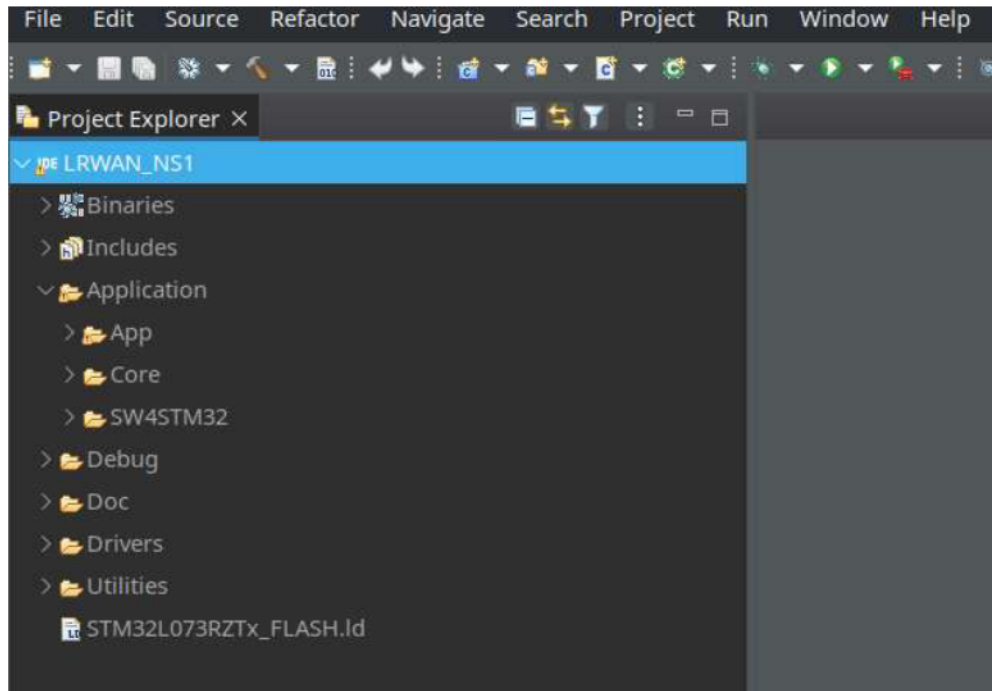


2. Dans le menu **C/C++ Build**, cliquez sur **Settings** → **MCU GCC Compiler** → **Warnings**.
3. Cochez la case « **Treat warnings as errors (-Werror)** ».

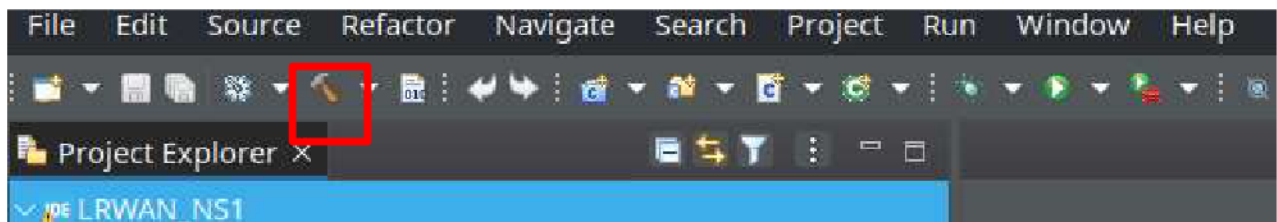


3. Compiler et exécuter pour la première fois

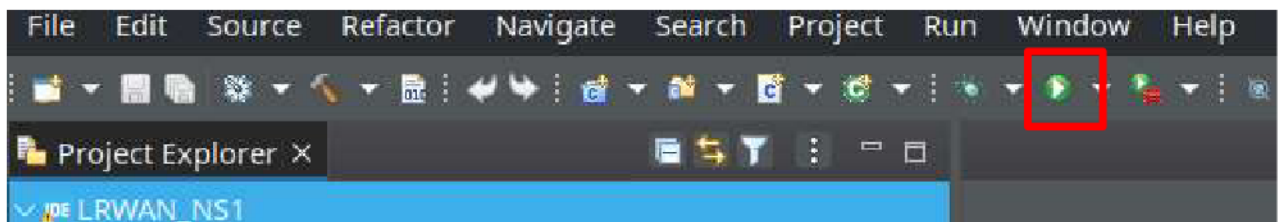
1. Normalement votre espace de travail doit ressembler à ceci :



Compilez le code (**build**, icône du marteau) :

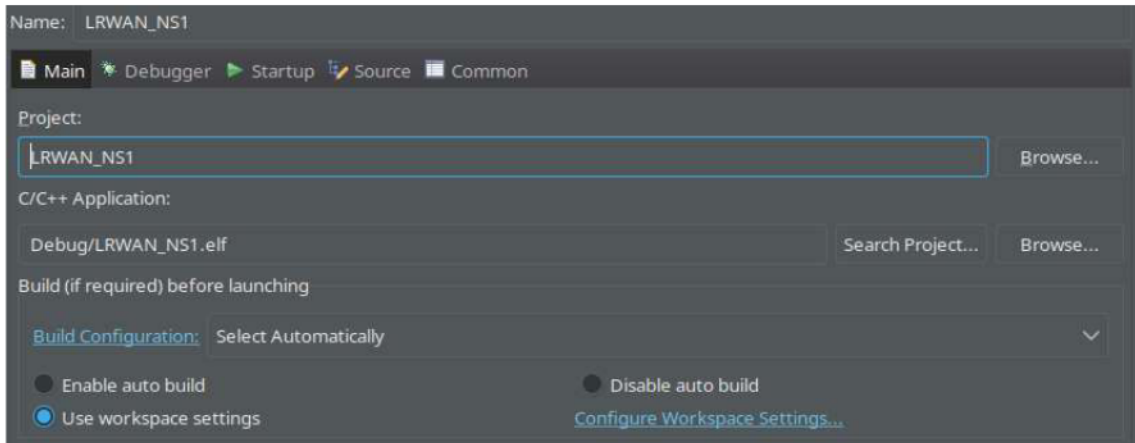


Lancez le code sur la carte P-NUCLEO-LRWAN3 (run, icône flèche à droite dans le rond vert)

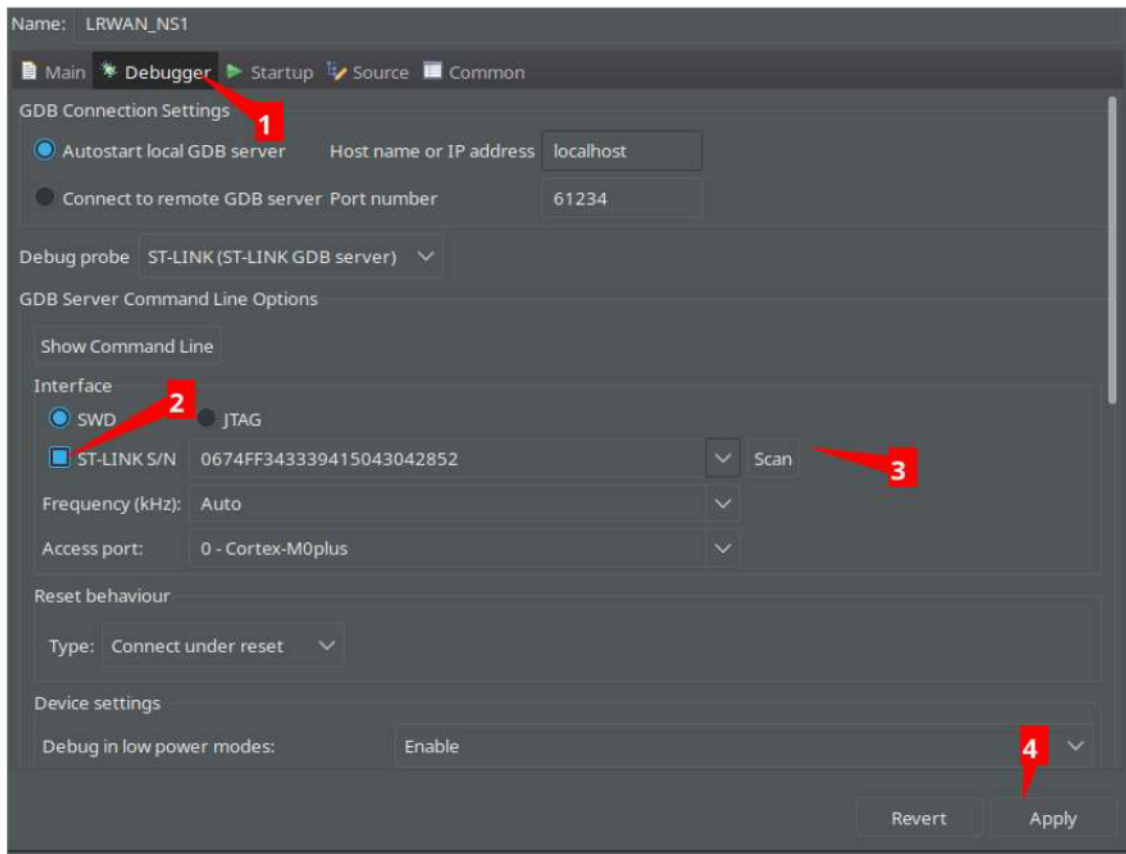


2. Si STM32Cube affiche une fenêtre avec plusieurs onglets comme ceci :
Ça veut dire qu'il ne trouve pas encore votre carte.

Pour résoudre le problème :



- i. Allez dans l'onglet « **Debugger** »
- ii. Dans l'encart Interface, cliquez sur le bouton à cocher **ST-LINK S/N**
- iii. Puis cliquez sur **Scan**
- iv. STM32CubeIDE trouve votre carte. le numéro de série au milieu va se remplir
- v. Validez avec le bouton **Apply**



3. Si STM32Cube vous embête avec le message :
ST-Link Server is required to launch the debug session

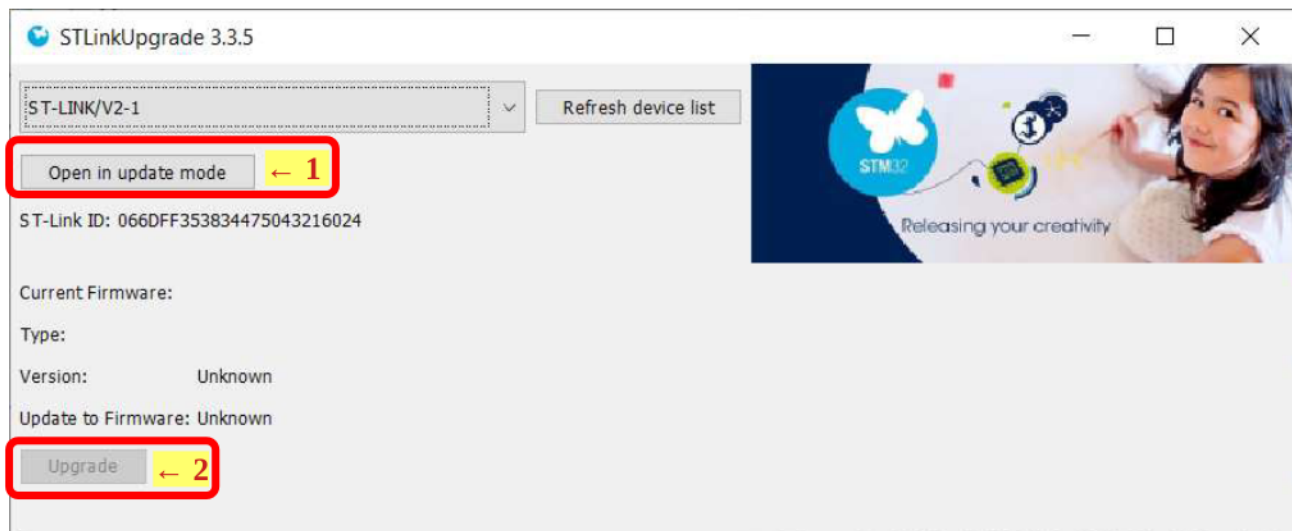


C'est qu'il lui manque le pilote pour parler à votre carte (ça peut arriver...)

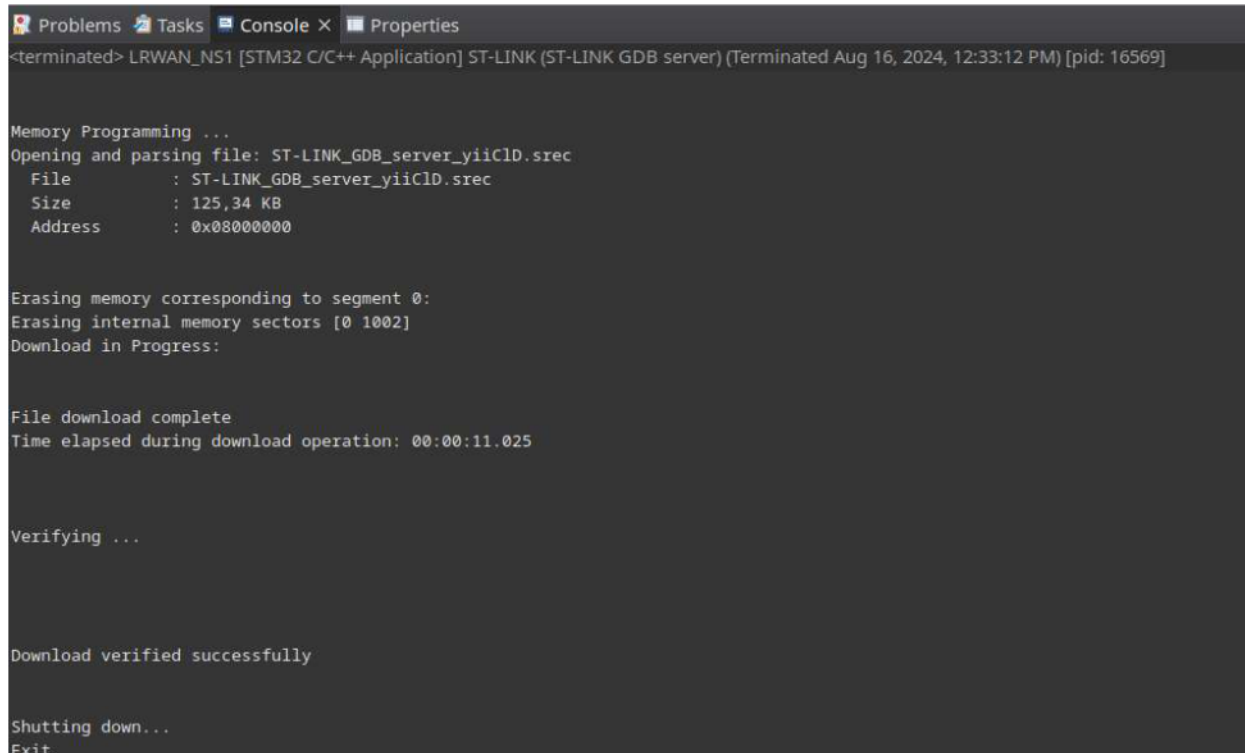
- i. Téléchargez-le ici (version **2.0.2**) :
<https://www.st.com/en/development-tools/st-link-server.html>
- ii. Installez-le pour votre plateforme (Mac, Windows, Linux...)

4. Si STM32Cube vous demande d'upgrader le firmware de votre carte:

- i. Acceptez 🙋
- ii. Dans l'écran suivant :
 1. Cliquez d'abord sur **Open in update mode**
 2. puis sur **Upgrade**
 3. Levez les mains et attendez que ça se finisse.



5. Dans l'onglet *console* vous verrez maintenant un log comme celui-ci .
Il confirme l'écriture du firmware dans la carte **P-NUCLEO-LRWAN3** :



```
Problems Tasks Console X Properties
<terminated> LRWAN_NS1 [STM32 C/C++ Application] ST-LINK (ST-LINK GDB server) (Terminated Aug 16, 2024, 12:33:12 PM) [pid: 16569]

Memory Programming ...
Opening and parsing file: ST-LINK_GDB_server_yiiC1D.srec
  File      : ST-LINK_GDB_server_yiiC1D.srec
  Size      : 125,34 KB
  Address   : 0x08000000

Erasing memory corresponding to segment 0:
Erasing internal memory sectors [0 1002]
Download in Progress:

File download complete
Time elapsed during download operation: 00:00:11.025

Verifying ...

Download verified successfully

Shutting down...
Exit.
```

6. Ouvrez un émulateur de terminal. Selon votre machine :
- CoolTerm pour Windows et Mac OS: <https://freeware.the-meiers.org/>
 - minicom pour Linux (sudo apt install minicom)
7. Configurez-le avec les paramètres de connexion **9600 8N1** sans contrôle de flux
8. Ouvrez le port série de débog créé par la **P-NUCLEO-LRWAN3**
- Windows : **COM1**, **COM2**, **COM3**...
 - Mac OS : **/dev/tty.usb0** à **9**, ou alors **usbmodemXXXX**...
 - Linux : **/dev/ttyACM0** à **9**..
9. Redémarrez la carte avec le grand bouton noir RESET en son centre. Vous devriez voir quelque chose comme ceci :

```
AT
+AT: OK
AT+ID=DevEui
+ID: DevEui, 8C:F9:57:20:00:00:F3:F4
AT+ID=AppEui
+ID: AppEui, F9:E9:AA:AA:AD:20:4A:72
AT+FDEFAULT
+FDEFAULT: OK
AT+LOWPOWER=AUTOON
+LOWPOWER: AUTOON
AT+DR=EU433
+DR: EU433
AT+MODE=LWOTAA
+MODE: LWOTAA
AT+ADR=OFF
+ADR: OFF
AT+DR=3
+DR: DR3
+DR: EU433 DR3 SF9 BW125K
AT+WDT=OFF
+WDT: OFF
```

QUESTION : Faites un screenshot de votre terminal, il devrait afficher des valeurs similaires.

10. Votre carte dispose de paramètres LoRa critiques à son fonctionnement : **DevEUI** et **AppEUI**. Ceux-ci sont spécifiques et uniques à chaque carte. En effet, ces valeurs sont officiellement attribuées par l'organisme mondial **IEEE Registration Authority** :

<https://standards.ieee.org/wp-content/uploads/import/documents/tutorials/eui.pdf>

QUESTION : Quelles sont vos valeurs **DevEUI** et **AppEUI** ?

11. Pour vous permettre de debugguer votre code, vous pouvez aussi rajouter votre propre log dans le terminal. Pour ce faire, utilisez `dbg_printf_send(const char* format,...)`, déclarée dans `lrwan_ns1_printf.h` . Par exemple :

```
dbg_printf_send("%s\n", __FUNCTION__);
```

QUESTION : Prouvez le bon fonctionnement de votre log d'initialisation avec un screenshot du terminal : vous devez y voir « **init done** »

12. 🎉 Génial ! Vous arrivez à afficher des logs, cette fonction est votre meilleure amie. Pour fêter ça, vous allez maintenant afficher des valeurs mesurées par les capteurs. Pour ce faire :

- Dans le projet, localisez la chaîne de caractères « **1. TODO LORA** »,
- De la structure **Sensor**, loggez la température avec une précision de deux décimales après la virgule.
- Indice : c'est un flottant, mais **%f** ne marchera pas (trop lourd, on vous explique plus bas dans le TP !).
Par contre, vous pouvez tout à fait prendre sa partie entière d'une part, et le multiplier par ?? d'autre part pour récupérer deux décimales, puis afficher ça comme deux entiers **%d.%d**

QUESTION : Prouvez que votre log d'affichage des valeurs de capteur marche avec un screenshot du terminal.

13. **OH NON**, rien ne s'affiche :(

QUESTION :

- Qui appelle la fonction de mesure des capteurs où vous êtes actuellement (**sensorMeasureData()**)? **Search** → **Files** est votre ami.
- Dans quel état devrait être cette fonction appelante pour invoquer **sensorMeasureData()**?
- Pourquoi n'y sommes-nous pas (pour le moment)?
- Pas grave, on peut faire sans ! Trouvez un case de machine d'état dans lequel le code passe forcément et appelez la fonction de mesure des capteurs.

14. **OMG** VOUS ARRIVEZ DÉJÀ À AFFICHER DES VALEURS DE TEMPÉRATURE !
ÇA MARCHE!!! 🎉🌡️

Sauf que... c'est toujours les mêmes, tout le temps et pour toutes les cartes, ce qui est statistiquement impossible 😞

En effet, pour le moment, les grandeurs mesurées sont initialisées à des *valeurs par défaut* car les capteurs **ne fonctionnent pas encore**... *Womp womp*.
Votre mission est de mettre en œuvre le code pour activer les capteurs et collecter les mesures requises.

4. Mise en œuvre d'un capteur, formatage des données

Repérer et intégrer l'en-tête du capteur de température

Nous travaillons sur la plateforme nommée `lrwan_ns1`.

Il y a un fichier d'API `.h` qui porte ce nom **ainsi que celui du capteur...** 😞

1. **QUESTION** : Qui et où est-il ? Comment l'inclure ?

Indice : cherchez « 2. **TODO LORA** »

Instancier sa variable de travail

2. **QUESTION** :

- Dans le `.h`, quelle fonction **initialise** le capteur de température ?
- Le premier argument définit **le nom du hardware du thermomètre**, le deuxième argument définit **sa variable de travail** (un « handle »).

- Rajoutez le code pour instancier le handle du thermomètre .

Indice où rajouter votre code : cherchez « 3. **TODO LORA** »

Initialiser le capteur

3. **QUESTION** : Appelez la fonction d'initialisation avec :

- argument 1 : nom hardware du capteur (`TEMPERATURE_ID_t`) = **HTS221_T_0**
- argument 2 : la variable de travail que vous avez instanciée ci-dessus.

Indice où rajouter votre code : cherchez « 4. **TODO LORA** »

Lire le capteur, vérifier la lecture

4. **QUESTION** :

- Étudiez l'API et rajoutez le code pour **lire** le capteur de température.
Indice où rajouter votre code: cherchez « 5. **TODO LORA** »

- Vérifiez le résultat depuis le `dbg_printf_send()` que vous avez rajouté à « 0. **TODO LORA** »

- **ARGH c'est toujours pas bon !** Qu'aurait-on oublié ?

Le capteur est-il par défaut **actif** ou **inactif** ?... Scrutez l'API et trouvez la fonction correspondante.

Au niveau de « 4. **TODO LORA** », rajoutez-y l'appel nécessaire.

Revérifiez que le résultat fonctionne cette fois-ci !

5. Convertir les données en valeurs à virgule fixe

Les données des capteurs sont fournies en nombres flottants. C'est étonnamment riche pour un capteur car... **Story time !**

En vérité, une couche d'abstraction a lieu dans les **pilotes** logiciels des capteurs. Elle convertit les valeurs mesurées par les capteurs en nombre flottants dans notre cas. Ceci est dû au fait que **les capteurs ne peuvent travailler qu'avec des nombres entiers (virgule fixe) et non en nombre « naturels »**, avec des décimales (donc pas des flottants).

Cette règle de conversion virgule fixe → valeur décimale dépend toujours du constructeur et du modèle de capteur. Ici elle ne nous intéresse pas, mais nous allons faire l'inverse (valeur décimale → virgule fixe) pour **envoyer les données de température depuis votre carte vers le serveur LoRa!** C'est la classe.

Pour cette conversion, la température intermédiaire sera exprimée en **déci-degrés** sur un **nombre entier signé de 2 octets**.

1. **QUESTION :**

- Rajoutez le code pour convertir la valeur du capteur de température dans le format souhaité.
Indice où rajouter votre code : localisez la chaîne de caractères « **6. TODO LORA** »
- Vérifiez que le résultat est cohérent depuis avec des logs sur la console via **dbg_printf_send()**

Générer les données envoyées par votre objet LoRa

- Dans le code, localisez les positions « 7. **TODO LORA** » et « 8. **TODO LORA** »
- C'est ici que vous préparerez vos données, de la manière suivante:
 - Vos données sont réparties en une liste de **blocs**
 - Chaque **bloc** est numéroté de manière **croissante**
 - Chaque **bloc** est identifié avec un **identifiant** `LPP_DATATYPE_XXX` (via une `enum`)
 - Les data de chaque **bloc** sont Big Endian : les poids forts d'abord, les poids faibles après. Exemple : le nombre 31250 en Big Endian s'écrit en deux octets : d'abord 0x7A puis 0x12.
- La structure globale de vos données est la suivante :

CHAMP DE BLOC	TAILLE EN OCTETS
Numéro de bloc	1 (valeur croissante pour chaque bloc)
Identifiant de bloc	1 (voir ordre ci-dessous)
Taille des données du bloc (octets)	1
Données	Variable selon taille (max 255 octets)

Structure d'un bloc de données

LPP_DATATYPE_FRAME_IDENTIFIER	String Identifiant de groupe (vos initiales, par ex. « BW_SJ_JI ») taille variable
LPP_DATATYPE_TEMPERATURE	Valeur intermédiaire de température 2 octets
LPP_DATATYPE_DIGITAL_INPUT	Niveau de batterie, en pourcentage 1 octet

Blocs de données ordonnées par leurs types

- Consignes de formatage de vos données:
 - Votre identifiant de groupe est constitué de la liste des initiales de ses membres séparées par un underscore (« _ »). Par exemple, pour le trinôme Bruce Willis / Samuel J. Jackson et Jeremy Irons, l'identifiant sera « **BW_SJ_JI** »
 - Pour récupérer le niveau de batterie, vous utiliserez `Lora_GetBatLevel()`, qui renvoie la batterie entre 0 et 254 sur un `uint32_t`. Dans vos données, vous les convertirez en pourcentage entier sur 8 bits.
 - Vérifiez que `Lora_GetBatLevel()` vous renvoie bien le status `ATCTL_RET_CMD_VDD` !

2. **QUESTION** : Montrez votre code de structuration de données.

6. Envoyer vos données au contrôleur LoRa

Vos données sont maintenant formatées, elles vont bientôt sortir de votre carte.

Le code du pilote LoRa se situe dans :

`Projects/NUCLEO-L073RZ/Applications/LoRaWAN/LoRaWAN_AT_Master/LoRaWAN/App/lor_driver.h` (pour les en-têtes) et `lor_driver.c` (pour l'implémentation).

La machine d'état (aka Finite State Machine, FSM) est dans la fonction `Lora_fsm()`.

Comme toute machine d'état classique, elle fonctionne avec un switch-case, chaque code de case permet de passer d'un état à l'autre.

L'état initial est `DEVICE_INIT`.

1. **QUESTION** : Il y a dans cet état, une fonction de timer qui démarre une **tâche précise**. Quel est le nom de la fonction relative à cette tâche ?
2. **QUESTION** : Dans quel nouvel état cette tâche met la machine d'état ?
3. **QUESTION** : Dans ce nouvel état, quelle est la fonction qui mesure effectivement les valeurs des capteurs ? (vous avez déjà donné la réponse, c'est pour voir si vous suivez)
4. **QUESTION** : Après que les valeurs sont mesurées, les valeurs ne sont pas envoyées par votre carte sur le réseau LoRa.
Il manque intentionnellement **l'appel à une dernière fonction** pour envoyer les données à la passerelle dans les environs.
Selon l'API de ce pilote LoRa, quel serait-il ?
Indice : on veut **envoyer** des **données** en **binaire** (en anglais ça donne...) Rajoutez l'appel et testez-le.

7. Faire transiter vos données vers le Cloud

Enregistrement de votre carte dans le cloud

Passez me voir avec les **DevEUI**, **AppEUI**, **AppKey** de votre carte.

Votre enseignant enregistrera votre Device sur son propre Network + App Server nommé **ChirpStack**. Il est puissant et open-source : <https://github.com/chirpstack/chirpstack>
Une fois configuré, celui-ci acceptera les trames provenant de votre carte via la gateway à proximité.

Ceci vous permettra ensuite d'accéder aux données des capteurs de votre carte depuis n'importe où sur *les internets* et en temps réel ! 😎

Recevoir vos données depuis le cloud

En retour, votre intervenant vous donnera un lien en protocole **WebSocket** (via **ws://...**). C'est un protocole permettant de transmettre des données simplement par le web, **sans fioritures** de présentation comme le fait HTTP (via **http://...**) et **connecté**, contrairement à HTTP.

Dans notre cas, ce lien WebSocket reflétera en temps réel les données que votre Device LoRa transmet à la gateway, puis au cloud, où que vous soyez.

Dans le lien de partage de documents pour ce TP, vous trouverez une application appelée **chirpstack_mqtt_over_ws.py**.

C'est un programme Python capable de recevoir des données par protocole WS. Vous aurez peut-être besoin d'installer le package **pip websocket-client**

1. **QUESTION** : adaptez le code de ce client à votre adresse WS.
Prouvez le bon fonctionnement en affichant le payload brut reçu par le cloud depuis votre propre carte LoRa.
2. **QUESTION** : Décodez votre trame de données pour retrouver :
 - votre préfixe de groupe
 - la valeur décimale du capteur de température
 - Le pourcentage de batterie
3. Avec la bibliothèque **Python Matplotlib**, réalisez les graphes des courbes de vos capteurs en temps réel.

Pour installer matplotlib: <https://matplotlib.org/stable/install/index.html>

Quelques exemples de mise en œuvre de **Matplotlib**:

<https://python.doctor/page-creer-graphiques-scientifiques-python-apprendre>

QUESTION : Montrez le bon fonctionnement de votre code par l'adéquation des données reçues et des graphes créés avec des screenshots pertinents.

8. Pour les vrai.es : Recommencez avec les capteurs d'humidité et de pression

Pour les vrai.es, le fun ne s'arrête jamais ! 🤖

Reprenez toutes les questions des parties **4 et 5**, en rajoutant les nouvelles grandeurs suivantes :

- La pression sera exprimée en daPa (**déca** Pascals), sur un **entier non signé de 2 octets**.
Pour l'Init(), le nom du hardware est **PRESSURE_SENSORS_AUTO**.
Rappel : le résultat fourni par le pilote du capteur est en Pascals flottants.
 - L'humidité sera exprimée en **demi-pourcentages**, sur un **entier non signé de 1 octet**.
Pour l'Init(), le nom du hardware est **HTS221_H_0**.
Rappel : le résultat fourni par le pilote du capteur est en pourcentages flottants.
1. **QUESTION** : Mettez en œuvre le nécessaire pour convertir les valeurs des capteurs d'humidité et de pression en valeurs à virgule fixe, puis les envoyer en LoRa selon le tableau de données finales exprimées dans l'ordre suivant:

LPP_DATATYPE_FRAME_IDENTIFIER	String d'identifiant de groupe taille variable
+LPP_DATATYPE_BAROMETER	Valeur intermédiaire de pression 2 octets
LPP_DATATYPE_TEMPERATURE	Valeur intermédiaire de température 2 octets
+LPP_DATATYPE_HUMIDITY	Valeur intermédiaire d'humidité 1 octet
LPP_DATATYPE_DIGITAL_INPUT	Niveau de batterie 1 octet

On rappelle la structure d'un bloc élémentaire de données :

CHAMP DE BLOC	TAILLE EN OCTETS
Numéro de bloc	1 (valeur croissante pour chaque bloc)
Identifiant de bloc	1 (voir ordre ci-dessous)
Taille des données du bloc (octets)	1
Données	Variable selon taille (max 255 octets)

Unité : Réseaux IoT
Yves Benabou
yves.benabou@esiee.fr

Félicitations, vous êtes vraiment arrivé.es à la fin 🎉 !

Nous espérons que ces TPs vous ont permis de découvrir l'univers passionnant de LoRa, de l'IoT en général et de sa chaîne de transmission du circuit intégré jusqu'au serveur d'application sur le Cloud.

Nous vous souhaitons de maintenant regarder les concepts de smart house, smart buildings, smart cities d'un nouvel œil curieux, ouvert et amusé.